

UNIwersytet Kardynała Stefana Wyszyńskiego
w Warszawie

Wydział Matematyczno-Przyrodniczy
Szkoła Nauk Ścisłych

Jan Zakroczyński

Nr alb.: 126732

Kierunek studiów: Informatyka

FAscnn++: Rozwinięcie architektury Fast-SCNN o mechanizm
Fast Attention i efektywny moduł fuzji cech
w zastosowaniu segmentacji obrazów

Praca licencjacka wykonana
pod kierunkiem naukowym
dr inż. Robert Kłopotek

Warszawa, 2026

Spis treści

Wykaz skrótów	5
Streszczenie	6
Rozdział 1. Wstęp	7
Rozdział 2. Przedstawienie problemu badawczego	8
2.1. Definicja problemu	8
2.2. Znaczenie ograniczeń wydajnościowych	8
2.3. Podstawy teoretyczne	8
2.3.1. Sieci konwolucyjne	8
2.3.2. Zaawansowane bloki architektoniczne	8
2.3.3. Samouwaga (Self-Attention) i bloki nielokalne	9
2.3.4. Segmentacja semantyczna, panoptyczna i instancyjna	10
2.3.5. Klasyczne modele segmentacji	10
2.3.6. Typy architektur	10
2.4. Przegląd architektur efektywnych i mechanizmów uwagi	11
2.5. Luka badawcza	11
2.6. Sformułowanie problemu badawczego	12
Rozdział 3. Implementacja	13
3.1. Architektura systemu i struktura projektu	13
3.2. Technologie i wymagania systemowe	13
3.3. Opis kluczowych komponentów	14
3.3.1. Moduł danych i augmentacja	14
3.3.2. Fabryka modeli – ModelBuilder	14
3.3.3. Pętla treningowa i ewaluacja	14
3.4. Konfiguracja i automatyzacja eksperymentów	14
3.5. Interfejs użytkownika (CLI)	15
Rozdział 4. Metodologia badań	16
4.1. Cel badań i pytania badawcze	16
4.2. Zbiór danych	16
4.3. Modele uwzględnione w eksperymentach	18
4.4. Warunki środowiska eksperymentalnego	18
4.5. Miary oceny modeli	19
4.6. Plan procesu badawczego	20
4.6.1. Etap 1: Inicjalizacja architektur referencyjnych	20
4.6.2. Etap 2: Ewolucja modelu FAscnn++	20

4.6.3. Etap 3: Walidacje końcowe i śledzenie zmian (Ablation)	20
4.7. Zasady rzetelności eksperymentalnej	21
Rozdział 5. Wyniki eksperymentów	22
5.1. Sposób prezentacji wyników	22
5.2. Wyniki modeli bazowych	22
5.2.1. Zestawienie wyników eksperymentalnych	22
5.3. Wyniki kolejnych wariantów FAscnn++	22
5.3.1. Analiza modelu FAscnn++_V18	23
5.4. Badania ablacyjne i ewolucja architektury	23
5.5. Wpływ rozdzielczości obrazu na wyniki treningu i inferencji	24
5.6. Analiza kompromisu pomiędzy jakością a szybkością inferencji	24
5.7. Analiza jakościowa segmentacji w obrębie klas	24
5.8. Porównanie z rozwiązaniami z literatury	26
5.9. Wnioski z przeprowadzonych eksperymentów	28
Rozdział 6. Podsumowanie	29
6.1. Stopień realizacji celu pracy	29
6.2. Weryfikacja hipotezy badawczej	29
6.3. Najważniejsze osiągnięcia pracy	29
6.4. Ograniczenia warsztatu projektowego	30
6.5. Kierunki dalszych prac	30
Bibliography	32
Dodatek A. Krzywe uczenia i wykresy szczegółowe metryk	36
Dodatek B. Przykładowe wyniki segmentacji modelu FAscnn++ V18	41

Wykaz skrótów

- ADAS** *Advanced Driver Assistance Systems* – zaawansowane systemy wspomagania kierowcy
- AI** *Artificial Intelligence* – sztuczna inteligencja
- AMP** *Automatic Mixed Precision* – automatyczna precyzja mieszana
- ASPP** *Atrous Spatial Pyramid Pooling* – przestrzenne piramidalne łączenie atrusowe
- CBAM** *Convolutional Block Attention Module* – splotowy moduł uwagi blokowej
- CLI** *Command Line Interface* – interfejs wiersza poleceń
- CNN** *Convolutional Neural Network* – konwolucyjna sieć neuronowa
- CPU** *Central Processing Unit* – główna jednostka obliczeniowa (procesor)
- CRF** *Conditional Random Field* – warunkowe pola losowe
- DSCConv** *Depthwise Separable Convolution* – splot separowalny w głąb
- FCN** *Fully Convolutional Network* – w pełni konwolucyjna sieć neuronowa
- FFM** *Feature Fusion Module* – moduł fuzji cech
- FN** *False Negative* – fałszywie negatywna detekcja (fałszywe odrzucenie)
- FP** *False Positive* – fałszywie pozytywna detekcja (fałszywy alarm)
- FPN** *Feature Pyramid Network* – piramidalna sieć cech
- FPS** *Frames Per Second* – liczba klatek na sekundę
- GFLOPs** *Giga Floating Point Operations* – miliardy operacji zmiennoprzecinkowych (miara złożoności modelu)
- GPU** *Graphics Processing Unit* – jednostka przetwarzania graficznego
- IoT** *Internet of Things* – Internet rzeczy
- IoU** *Intersection over Union* – miara pokrycia predykcji z wzorcem (przecięcie przez sumę)
- LR** *Learning Rate* – współczynnik uczenia
- mIoU** *mean Intersection over Union* – uśredniona wartość miary IoU dla wszystkich klas
- MLP** *Multi-Layer Perceptron* – wielowarstwowy perceptron
- NAS** *Neural Architecture Search* – automatyczne wyszukiwanie architektury sieci neuro-
nowej
- PA** *Pixel Accuracy* – dokładność pikselowa
- R&D** *Research and Development* – badania i rozwój
- SGD** *Stochastic Gradient Descent* – stochastyczny spadek wzdłuż gradientu
- SOTA** *State of the Art* – najnowocześniejsze osiągnięcia w dziedzinie
- TP** *True Positive* – prawdziwie pozytywna detekcja
- UAV** *Unmanned Aerial Vehicle* – bezzałogowy statek powietrzny (dron)

Streszczenie

Praca przedstawia projekt i ocenę architektury FAscnn++ do semantycznej segmentacji obrazów w czasie rzeczywistym. Głównym celem badań jest analiza kompromisu pomiędzy dokładnością segmentacji a wydajnością obliczeniową lekkiej architektury wielostrumieniowej rozszerzonej o moduły uwagi. Eksperymenty przeprowadzono na zbiorze danych Cityscapes. Część teoretyczna obejmuje przegląd ograniczeń modeli czasu rzeczywistego oraz architektur referencyjnych. W części praktycznej opisano implementację sieci, konfigurację środowiska testowego oraz ewaluację modelu FAscnn++. Analizie poddano kluczowe metryki jakości (mIoU, dokładność pikselowa) oraz wskaźniki przepustowości (FPS, opóźnienie inferencji).

Słowa kluczowe: segmentacja semantyczna; FAscnn++; Cityscapes; architektury lekkie; czas rzeczywisty.

Rozdział 1

Wstęp

Semantyczna segmentacja obrazów jest kluczowym problemem w dziedzinie widzenia maszynowego [31]. Klasyfikacja obiektów na poziomie pojedynczych pikseli umożliwia precyzyjną interpretację przestrzenną sceny. Ma to krytyczne znaczenie dla działania systemów ADAS [11], autonomicznych bezzałogowców [32] oraz robotyki mobilnej [13].

W zastosowaniach czasu rzeczywistego (np. autonomiczna nawigacja w scenach miejskich) modele wizyjne muszą rozpoznawać wiele złożonych klas (infrastruktura drogowa, piesi, pojazdy) z ułamkowym opóźnieniem. Głównym wyzwaniem projektowym w tym obszarze jest optymalizacja kompromisu pomiędzy dokładnością predykcji a narzutem obliczeniowym sieci splotowej.

Głębokie modele segmentacyjne o wysokiej precyzji, takie jak SETR [54] czy Mask2Former [3], charakteryzują się potężną złożonością matematyczną. Całkowicie dyskwalifikuje je to z bezpośrednich wdrożeń brzegowych (edge computing). Praktycznym rozwiązaniem są wysoce zoptymalizowane, lekkie architektury splotowe, w tym ENet [33] i Fast-SCNN [37], ukierunkowane na drastyczną minimalizację liczby parametrów. Poprawa precyzji działania takich architektur, przy zachowaniu bezkompromisowej przepustowości, pozostaje w centrum aktualnych prac badawczych.

Celem pracy jest projekt, oprogramowanie i rygorystyczna ewaluacja rozszerzeń dla architektury Fast-SCNN. Skonstruowano model FAscnn++, łączący strukturę wielostrumieniową z wydajnymi mechanizmami uwagi. Przygotowane rewizje sieci przetestowano i porównano analitycznie z modelami referencyjnymi na zbiorze Cityscapes [5].

Hipoteza badawcza zakłada, że integracja modułów uwagi ze sprawdzoną topologią splotową w modelu FAscnn++ znacząco zwiększy metrykę dokładności (mIoU) względem innych modeli klasy lekkiej. Zostanie to osiągnięte przy utrzymaniu przepustowości klatek (FPS) kompatybilnej z systemami o reżimie czasu rzeczywistego. Poprawność tej tezy zweryfikowano wykorzystując ustrukturyzowany pakiet miar jakości (mIoU, Pixel Accuracy) oraz miar wydajności (FPS, latency, GFLOPs, liczba parametrów).

Struktura dysertacji: Rozdział 2 formułuje wprost problem badawczy. Rozdział 3 dokumentuje autorską implementację programową. W rozdziale 4 sformalizowano metodologię i warunki środowiska testowego. Rozdział 5 to szczegółowa i analityczna ewaluacja uzyskanych wyników. Pracę zamyka wykaz kluczowych konkluzji (Rozdział 6).

Rozdział 2

Przedstawienie problemu badawczego

2.1. Definicja problemu

Problem badawczy dotyczy semantycznej segmentacji scen miejskich w czasie rzeczywistym. Zadanie polega na przypisaniu każdemu pikselowi obrazu o wymiarach $H \times W$ etykiety klasy semantycznej. Kluczowym aspektem pracy jest optymalizacja kompromisu pomiędzy dokładnością predykcji a narzutem obliczeniowym. Szybkość wnioskowania determinuje użyteczność modelu w systemach takich jak ADAS czy robotyka mobilna. Wysoka przepustowość (FPS) umożliwia szybszą nawigację i krótszy czas reakcji na przeszkody.

2.2. Znaczenie ograniczeń wydajnościowych

Wiele współczesnych modeli segmentacji osiąga wysoką dokładność kosztem dużej liczby parametrów, obciążenia pamięci oraz długiego czasu inferencji. Ogranicza to ich wdrożenie na platformach brzegowych (Edge AI). Z tego względu ewaluacja obejmuje zarówno jakość predykcji, jak i opóźnienie (latency). W zastosowaniach takich jak ADAS, bezzałogowce i robotyka, opóźnienie stanowi parametr krytyczny. Standardy inżynierskie często definiują czas rzeczywisty jako przepustowość powyżej 30 FPS [9]. W aplikacjach wymagających dynamicznej pętli sprzężenia zwrotnego (np. *visual servoing*) celuje się w 60 Hz [6]. Wymagania te zależą ściśle od użytego sprzętu i dynamiki otoczenia.

2.3. Podstawy teoretyczne

2.3.1. Sieci konwolucyjne

Podstawowym modułem w modelach wizyjnych są konwolucyjne sieci neuronowe (CNN), zbudowane z warstw splotowych i nieliniowych aktywacji. W niniejszej pracy analizowane są wyłącznie warianty architektur splotowych optymalizowane pod kątem wysokiej wydajności.

2.3.2. Zaawansowane bloki architektoniczne

Optymalizacja architektur CNN wymaga modyfikacji strukturalnych, z których kluczowe to konwolucje separowalne, bloki typu *bottleneck* oraz przetwarzanie wieloskalowe.

Konwolucje separowalne (Depthwise Separable Convolutions) Tradycyjny spłot (jądro $D_K \times D_K$, M kanałów wejściowych, N wyjściowych, rozdzielczość mapy $D_F \times D_F$) wiąże się z kosztem obliczeniowym:

$$C_{std} = D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F$$

Konwolucja separowalna [19], [4] rozdziela tę operację na spłot przestrzenny (*depthwise*) i punktowy (*pointwise*). Koszt wynosi wówczas:

$$C_{sep} = D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F$$

Mechanizm ten drastycznie kompresuje model, osiągając współczynnik redukcji operacji równy $\frac{1}{N} + \frac{1}{D_K^2}$.

Moduły wąskiego gardła (Bottleneck) W sieciach takich jak ResNet [14] moduły wąskiego gardła zapobiegają eksplozji parametrów w głębokich warstwach. Dane $x \in \mathbb{R}^{H \times W \times C}$ podlegają rzutowaniu do niższej wymiarowości (kompresja), spłotowi przestrzennemu, a następnie ekspansji kanałowej:

$$F(x) = W_{1 \times 1}^{out} * \sigma \left(W_{3 \times 3} * \sigma \left(W_{1 \times 1}^{in} * x \right) \right)$$

Liczba kanałów filtru $W_{1 \times 1}^{in}$ jest znacznie mniejsza niż C , co pozwala na oszczędność pamięci i mocy obliczeniowej.

Mechanizmy uwagi i ich koszt obliczeniowy

W konwencjonalnych sieciach CNN wagi filtrów są statyczne niezależnie od lokalizacji. Mechanizmy uwagi (*attention*) umożliwiają dynamiczne ważenie cech, wzmacniając zdolności reprezentacyjne sieci przy minimalnym pogłębianiu architektury.

Uwaga kanałowa i przestrzenna Moduł *Squeeze-and-Excitation* (SE) wdrożony w sieciach SENet [20] agreguje informację przestrzenną poprzez *Global Average Pooling*. Wynik przetwarzany jest przez dwuwarstwowy perceptron (MLP), generując wektor wag $\alpha \in \mathbb{R}^C$. Kanały macierzy cech X są następnie skalowane:

$$X'_c = \alpha_c \cdot X_c$$

Moduł CBAM [46] rozszerza to rozwiązanie, aplikując sekwencyjnie uwagę kanałową oraz przestrzenną. Koszt tych mechanizmów to ułamek procenta całkowitej wartości GFLOPs modelu.

2.3.3. Samouwaga (Self-Attention) i bloki nielokalne

Koncepcja architektury Transformer [43] została przeniesiona do sieci CNN w formie bloków nielokalnych (*Non-local Neural Networks*) [45]. Dla wektora wejściowego rzutowanego na macierze zapytań (Q), kluczy (K) i wartości (V), operacja atencji oblicza wagowaną sumę:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

gdzie d_k to współczynnik skalujący.

Złożoność obliczeniowa samouwagi Złożoność klasycznej samouwagi zależy od rozdzielczości obrazu. Dla mapy cech o liczbie pikseli $N = H \times W$ i wymiarze C , obliczenie macierzy podobieństwa QK^T wymaga $\mathcal{O}(N^2C)$ operacji. Całkowita złożoność wynosi [8]:

$$\mathcal{O}(N^2C) = \mathcal{O}(H^2 \cdot W^2 \cdot C)$$

Zależność kwadratowa od rozdzielczości wyklucza zastosowanie tego mechanizmu na wczesnych etapach sieci (duże H i W). Narzędzie to stosuje się po wstępnej redukcji cech lub z wykorzystaniem uwagi lokalnej (np. Swin Transformer [26]).

Mechanizmy uwagi w segmentacji Uwaga globalna poprawia dokładność segmentacji poprzez modelowanie rozległego kontekstu sceny [22]. Ograniczeniem pozostaje operacja Softmax z kwadratową złożonością przestrzenną $\mathcal{O}(n^2c)$ (gdzie $n = H \times W$) [22]. Blokując to implementację klasycznej uwagi w systemach czasu rzeczywistego, dla których analiza pełnej klatki wideo jest konieczna.

Rozwiązania dla ograniczeń sprzętowych Skuteczną alternatywą jest mechanizm szybkiej uwagi (*Fast Attention*) [22]. Eliminacja normalizacji Softmax na rzecz miary podobieństwa kosinusowego pozwala na rearanżację kolejności mnożenia macierzy. Złożoność operacji maleje do wymiaru liniowego $\mathcal{O}(nc^2)$ [22]. W zadaniach wizyjnych ($n \gg c$) mechanizm ten ekstrahuje nielokalny kontekst przy ułamkowym koszcie względem klasycznej uwagi.

2.3.4. Segmentacja semantyczna, panoptyczna i instancyjna

Wizja komputerowa wyróżnia segmentację semantyczną (klasyfikacja pikseli) [27], instancyjną (identyfikacja odrębnych obiektów) [15] oraz panoptyczną (połączenie obu metod) [23]. Praca koncentruje się na segmentacji semantycznej, dostarczającej ogólnego opisu sceny, wymaganego do autonomicznej nawigacji w reżimie niskich opóźnień inferencji.

2.3.5. Klasyczne modele segmentacji

Pionierskim rozwiązaniem w dziedzinie są sieci FCN [27]. Inne klasyczne struktury, jak U-Net [38], SegNet [1], DeepLab [2] oraz PSPNet [53], osiągają dużą celność. Z uwagi na wysokie zapotrzebowanie na GFLOPs są nieużyteczne na platformach brzegowych. Ich dedykowanym zamiennikiem dla systemów mobilnych stała się sieć ICNet [52].

2.3.6. Typy architektur

Enkoder-dekoder Architektury sekwencyjne (np. U-Net [38]) realizują rekonstrukcję z użyciem połączeń omijających (*skip connections*). Koszt obliczeniowy wymusza ich odrzucenie z aplikacji czasu rzeczywistego.

Wielogłęziowe i wielorozdzielcze Systemy w rodzaju ICNet [52] czy Lite-HRNet [50] przetwarzają obraz równolegle w wielu rozdzielczościach. Ekstrakcja kontekstu odbywa się

na silnie skompresowanych mapach, a odtwarzanie detali na płytkich warstwach o szerokiej przestrzeni.

Dwuścieżkowe i wielościeżkowe Modele brzegowe opierają się na dekompozycji przetwarzania. Sieć BiSeNet [49], [48] rozdziela ścieżkę przestrzenną i kontekstową. Konstrukcje DDRNet [17] oraz PIDNet [47] rozbudowują tę ideę o gęstą fuzję sygnału. Takie ujęcie stanowi architektoniczny fundament proponowanego modelu FAscnn++.

2.4. Przegląd architektur efektywnych i mechanizmów uwagi

Pierwsza generacja i podejścia wieloskalowe Wczesne modele (ENet [33], ESPNetv2 [30]) optymalizowały obciążenie sprzętowe kompresując kanały i aplikując asymetryczne sploty. Kosztem była utrata detali. Architektura Fast-SCNN [36] z wbudowanym modulem kaskadowym (*Learning to Downsample*) ustaliła standard wydajności (123 FPS) w formacie 1024×2048 . Fast-SCNN służy jako główny punkt referencyjny niniejszej pracy.

Nowoczesne architektury wielościeżkowe Współczesne sieci SOTA odseparowują ścieżkę detali od kontekstu (BiSeNetV2, STDC2). DDRNet-23-slim pozwala na 77,4% mIoU przy 102 FPS. Architektura PIDNet-S notuje 78,6% mIoU (93,2 FPS). Ich implementacja dla urządzeń o ścisłym reżimie energetycznym pozostaje niemożliwa. Nowsze modele ograniczają opóźnienia poprzez skracanie dekodery (PP-LiteSeg-T). Model LCNet3_7 redukuje wielkość do 0,51 miliona parametrów zachowując 73,8% mIoU. Układ ISANet podnosi mIoU do 76,7% (1,37 M parametrów). Nowatorska sieć GCNet poprzez reparametryzację uzyskuje 77,3% mIoU i rekordowe 193,3 FPS w natywnej rozdzielczości 1024×2048 .

Transformery wizyjne, szybka uwaga (Fast Attention) i modele hybrydowe Transformery wizyjne redukują operacje uwagi wykorzystując warianty strukturalne: EfficientViT [25], TopFormer-T, SeaFormer-S/T oraz systemy hybrydowe (MobileViT [28]). Sieć LMIINet łączy splot i atencję, zapewniając 72,0% mIoU na zaledwie 0,72 M parametrach i przy obciążeniu 11,74 GFLOPs. Model RTLinearFormer (2025) stosuje uwagę liniową, notując płynność 66,7 FPS przy 78,41% mIoU. Algorytmem krytycznym dla badanego obszaru jest mechanizm modelu FANet [21]. Zmiana operacji Softmax na *cosine similarity* zapewnia liniową kalkulację uwagi ($\mathcal{O}(nc^2)$). Kluczowe dane ewaluacyjne dla modelu FAscnn++_V18 i sieci referencyjnych zamieszczono w Tabeli 5.6.

2.5. Luka badawcza

Rynek systemów wizyjnych jest silnie spolaryzowany. Wysoce precyzyjne sieci (PIDNet [47], DDRNet [16], GCNet) wymagają infrastruktury wielkości 20–100 GFLOPs, wykluczając instalacje w lekkich jednostkach (UAV). Z kolei modele z segmentu *ultra-lightweight* (PP-LiteSeg-T1 [35], STDC2-Seg50 [10], LCNet3_7 [40]) osiągają płynność kosztem kompresji klatki do formatów podrzędnych (np. 512×1024). Generuje to utratę kluczowych detali krawędzi i spadki precyzji w obrębie małych obiektów.

Wymuszenie obsługi klatek w rozmiarze natywnym (1024×2048) prowadzi do utraty parametrów pracy. TopFormer-T [51] oferuje ~ 100 FPS (RTX 3090) z bardzo słabym wynikiem 70,70% mIoU. Sieć SeaFormer-S [44] gwarantuje 76,10% mIoU, ale z płynnością jedynie 7,6 FPS na urządzeniach mobilnych.

Zidentyfikowana luka sprzętowa i algorytmiczna dowodzi braku architektury konwulucyjnej, która jednocześnie:

- **Przełamuje kompromis rozdzielczości:** przetwarza niezmodyfikowane wideo jakości 1024×2048 zachowując zapotrzebowanie do 10 GFLOPs i budżet do 1,2 miliona parametrów.
- **Gwarantuje dokładność krytyczną:** osiąga jakość przekraczającą 65% mIoU na zbiorze Cityscapes przy zachowaniu rozdzielczości rzędu natywnego.
- **Oferuje elastyczne środowisko dla badań (Edge AI):** posiada wdrożenia pozwalające na bezkompromisowe badania ablacyjne dla nowo projektowanych modułów sprzętowych.

2.6. Sformułowanie problemu badawczego

Celem naukowym pracy jest wdrożenie elastycznej architektury wpasowującej się w powyższą lukę. Problem sprowadzono do weryfikacji pytań badawczych:

- Czy topologiczna poprawa układów z rodziny Fast-SCNN (jako wariant FAscnn++) oferuje zoptymalizowany stosunek mIoU do GFLOPs dla predykcji pełnoklatkowej (1024×2048) w odniesieniu do sieci *ultra-lightweight*?
- Jak zdefiniowane mechanizmy splotowe wpływają na narzut strukturalny i jakość klasyfikacji poszczególnych klas?
- Czy dodatkowe zużycie pasma pamięci wbudowanej (*memory bandwidth*) w ramach mechanizmów Fast Attention uniemożliwi płynne działanie sieci utrzymanej w restrykcjach < 10 GFLOPs?

Rozdział 3

Implementacja

3.1. Architektura systemu i struktura projektu

Zaprojektowany system posiada strukturę modułową, co ułatwia integrację nowych modeli i modyfikację parametrów treningowych bez ingerencji w rdzeń aplikacji. Drzewo katalogów odzwierciedla podział na moduły logiczne:

- `src/data/` – definicje zbiorów danych (m.in. `Cityscape.py`) oraz logika transformacji i augmentacji.
- `src/model_architecture/` – moduł sieci neuronowych. Podkatalog `FAscnn++/` grupuje warianty architektoniczne (od V1 do V31). Plik `model_factory.py` dostarcza klasę `ModelBuilder` opartą na wzorcu projektowym fabryki.
- `src/utils/` – komponenty pętli treningowej (`train.py`), procedury ewaluacyjne (`test.py`), implementacje funkcji strat (`loss.py`) oraz kalkulatory metryk (`metrics.py`).
- `src/__main__.py` – główny punkt wejścia aplikacji (CLI), integrujący moduły i parsujący konfigurację.
- `visualisation/` – zbiór skryptów generujących wizualizacje predykcji oraz wykresy ewaluacyjne.

3.2. Technologie i wymagania systemowe

Aplikację zaimplementowano w języku Python [42] (wersja ≥ 3.8). Silnikiem obliczeniowym dla sieci neuronowych jest biblioteka PyTorch [34] (wersja 2.1), gwarantująca akcelerację sprzętową GPU poprzez biblioteki CUDA.

Kluczowe zależności środowiskowe obejmują:

- `torch` (2.10.0) – framework obliczeniowy operujący na tensorach.
- `numpy` (2.4.1) – wysokowydajne operacje macierzowe.
- `matplotlib` (3.10.8) – silnik wizualizacji danych.
- `PyYAML` (6.0.3) – parser plików konfiguracyjnych.
- `einops` (0.8.2) – elastyczne operacje tensorowe, wykorzystywane w implementacji mechanizmów uwagi.
- `tqdm` (4.67.1) – wskaźniki postępu dla CLI.
- `tensorboard` (2.20.0) – platforma logowania metryk treningowych.

3.3. Opis kluczowych komponentów

3.3.1. Moduł danych i augmentacja

Klasa `CityscapesDataModule` (`src/data/Cityscape.py`) implementuje potok ładowania zestawu Cityscapes. Do jej zadań należą:

- mapowanie identyfikatorów zbioru źródłowego na 19 klas treningowych,
- augmentacja strumieniowa (ang. *on-the-fly*): losowe skalowanie, przycinanie (*random crop*), lustrzane odbicia horyzontalne (*horizontal flip*) i modyfikacje jasności,
- transformacja do formatu tensora i normalizacja wejścia w oparciu o statystyki bazy ImageNet.

3.3.2. Fabryka modeli – ModelBuilder

Klasa `ModelBuilder` (`src/model_architecture/model_factory.py`) odpowiada za instancjonowanie architektur sieci. Abstrakcja ta obsługuje modele referencyjne (ENet [33], Fast-SCNN [36]) oraz wszystkie warianty rodziny FAsCNN++. Instancjonowanie konkretnej topologii wymaga jedynie modyfikacji odpowiedniego parametru w pliku konfiguracyjnym.

3.3.3. Pętla treningowa i ewaluacja

Funkcja `train_main` (`src/utils/train.py`) obsługuje cykl życia procesu uczenia:

- alokację optymalizatora (SGD z parametrem *momentum*) i harmonogramu współczynnika uczenia (`PolyLR`),
- akcelerację obliczeń dzięki zastosowaniu mieszanej precyzji zmiennoprzecinkowej (AMP – *Automatic Mixed Precision*),
- synchronizację z platformą TensorBoard w celu logowania przebiegu funkcji straty oraz metryk mIoU i Pixel Accuracy,
- automatyczną serializację punktów kontrolnych modelu (wagi *best.pt* i *last.pt*).

3.4. Konfiguracja i automatyzacja eksperymentów

Zarządzanie parametrami eksperymentów realizowane jest za pośrednictwem pliku `config.yaml`, co zapewnia reprodukowalność procedur ewaluacyjnych. Struktura konfiguracji dzieli się na sekcje:

- **training**: hiperparametry optymalizacyjne (LR, batch size, liczba epok), definicje ścieżek dostępu i warianty augmentacji.
- **test**: rozdzielczość wejściowa dla ewaluacji i liczba generowanych wizualizacji.
- **model**: deskryptor wybranej topologii sieci oraz liczba obsługiwanych klas.
- **patch**: parametry opcjonalnego kafelkowania klatek (ang. *patching*).

Przygotowano również zbiór skryptów powłoki (`.sh`) automatyzujących rutynowe procesy:

- `setup.sh` – budowa środowiska wirtualnego (*venv*) i instalacja bibliotek.
- `run_train.sh` – inicjalizacja potoku treningowego z domyślną parametryzacją.

- `run_test.sh` – egzekucja walidacji skuteczności modelu na dedykowanym zbiorze danych.

3.5. Interfejs użytkownika (CLI)

Aplikacja udostępnia interfejs wiersza poleceń (CLI), pozwalający na nadpisywanie parametrów środowiskowych. Przykłady wywołań:

- **Trenowanie modelu:**

```
python -m src train -config config.yaml -model FAscnn+_V13 -batch-size 8
```

- **Testowanie i ewaluacja:**

```
python -m src test -config config.yaml -model FAscnn+_V13
```

Wykorzystanie biblioteki `argparse` umożliwia nadpisywanie wartości parametrów z pliku YAML bezpośrednio w argumentach wywołania, co znacznie usprawnia proces optymalizacji hiperparametrów.

Rozdział 4

Metodologia badań

4.1. Cel badań i pytania badawcze

Głównym celem pracy jest implementacja i ewaluacja optymalizacji dla referencyjnej architektury Fast-SCNN. Wprowadzone ulepszenia (oznaczone jako model FAscnn++) dedykowane są systemom segmentacji semantycznej czasu rzeczywistego. Badania koncentrują się na maksymalizacji dokładności klasyfikacji (mIoU) przy jednoczesnym utrzymaniu wysokiej przepustowości (FPS) i niskiego opóźnienia inferencji (latency).

W ramach eksperymentów zestawia się wydajność modeli z rodziny FAscnn++ z referencyjnymi architekturami ENet oraz Fast-SCNN, zachowując ujednolicone warunki uczenia i tożsame środowisko sprzętowe.

Na podstawie analizy wyników sformułowano odpowiedzi na pytania badawcze:

1. Czy wdrożenie zmodyfikowanej fuzji dwustrumieniowej w architekturze Fast-SCNN zwiększa precyzję segmentacji (mIoU) przy zachowaniu porównywalnego narzutu obliczeniowego?
2. W jakim stopniu mechanizm **FastAttention** zintegrowany ze ścieżką kontekstową przewyższa klasyczne moduły *Global Average Pooling* w klasyfikacji obiektów o dużej wariancji skali?

4.2. Zbiór danych

Eksperymenty oparto na zbiorze *Cityscapes Dataset* [5], stanowiącym standard w ewaluacji modeli wizyjnych dla systemów autonomicznych. Zbiór oferuje gęste adnotacje pikselowe obrazów zarejestrowanych w 50 miastach europejskich.

Charakterystyka i struktura zbioru Baza obejmuje 5000 obrazów w rozdzielczości 2048×1024 pikseli z precyzyjnymi maskami (*fine annotations*). Zestaw podzielono na:

- **Zbiór treningowy:** 2975 obrazów wykorzystywanych do optymalizacji wag sieci.
- **Zbiór walidacyjny:** 500 obrazów do monitorowania metryki mIoU i strojenia hiperparametrów.
- **Zbiór testowy:** 1525 obrazów z maskami niejawnymi, służących do końcowej ewaluacji po stronie serwera projektowego.

Klasyfikacja i trudności badawcze Procedura ewaluacyjna wyodrębnia 19 klas krytycznych dla nawigacji w ruchu miejskim. Obejmują one kategorie infrastruktury drogowej

(jezdnia, chodnik), uczestników ruchu (pieszy, rowerzysta), pojazdów (samochód, autobus) oraz elementów statycznych (budynki, ogrodzenie).

Głównym problemem analitycznym zestawu jest silne niezbalansowanie klas (*class imbalance*). Dominują klasy tła (droga, niebo, budynki), natomiast obiekty takie jak motocykl czy sygnalizacja świetlna zajmują ułamkową powierzchnię kadrów. Wymusza to zastosowanie na etapie treningu technik kompensujących dominację długiego ogona, aby umożliwić modelowi ekstrakcję cech rzadkich obiektów bez obniżenia przepustowości FPS.

Przetwarzanie i augmentacja danych W celu poprawy generalizacji zaimplementowano strumieniowy potok augmentacji danych (*on-the-fly*):

- **Losowe skalowanie:** ze współczynnikiem w zakresie od 0,5 do 2,0 rozdzielczości bazowej.
- **Przycinanie (Random Crop):** do docelowego formatu 1024×2048 pikseli (lub skompresowanego, zależnie od specyfikacji testu).
- **Odbicie lustrzane:** horyzontalne (*horizontal flip*) z prawdopodobieństwem 0,5.
- **Augmentacja Copy-Paste:** algorytm niwelujący problem *long tail*. System losowo wyodrębnia rzadkie klasy z obrazów dawców i implementuje je na tło docelowe. Operacja ta zwiększa ekspozycję mało reprezentowanych kategorii w procesie uczenia bez ingerencji w globalny kontekst.
- **Normalizacja:** zgodna ze średnią wyliczoną dla zbioru ImageNet, przyspieszająca konwergencję gradientu.

W trybie inferencji modele ewaluowano na pełnych klatkach 2048×1024 .

Kryteria doboru klas do augmentacji Copy-Paste Wyodrębnienie celów dla techniki Copy-Paste zrealizowano metodą ilościową (*Data-Driven Copy-Paste*). Dla klasy i wyznaczono metryki:

1. **Globalne prawdopodobieństwo wystąpienia** ($P_{glob,i}$), definiujące udział pikseli klasy w zbiorze treningowym:

$$P_{glob,i} = \left(\frac{C_i}{T_{pixels}} \right) \cdot 100\% \quad (4.1)$$

gdzie C_i to suma pikseli klasy i , a T_{pixels} oznacza łączną liczbę pikseli w zestawie uczącym.

2. **Warunkowe pokrycie obrazu** ($A_{cond,i}$), szacujące powierzchnię zajmowaną przez obiekt przy jego wystąpieniu:

$$A_{cond,i} = \left(\frac{C_i/N_i}{P_{img}} \right) \cdot 100\% \quad (4.2)$$

gdzie N_i to liczba wystąpień obrazów z klasą i , a P_{img} odpowiada liczbie pikseli na obraz.

Procedurę zaaplikowano z ograniczeniem progowym: $P_{glob,i} < 1,0\%$ i $A_{cond,i} < 1,0\%$. Skutkuje to kwalifikacją wyłącznie obiektów rzadkich i małowagarytowych (światła, znaki, motocykle, rowery). Takie ujęcie zapobiega modyfikacjom przestrzennym ingerującym w topologię otoczenia ustrukturyzowanego.

4.3. Modele uwzględnione w eksperymentach

Ocenie poddano zbiór następujących wariantów sieci neuronowych:

- architektury bazowe: Fast-SCNN, ENet (wraz ze skróconymi wariantami ENetv2, ENetv3),
- cykl ewolucyjny środowiska FAscnn++,
- docelowy model po przeprowadzeniu testów ablacyjnych (FAscnn++_V18).

Wykorzystanie ENet oraz Fast-SCNN pozwala na standaryzowane, weryfikowalne zestawienie wydajności względem dotychczasowych rozwiązań brzegowych.

4.4. Warunki środowiska eksperymentalnego

Proces badawczy zrealizowano w ujednoliconym środowisku obliczeniowym. Wdrożono rygorystyczną standaryzację hiperparametrów treningowych w celu zapewnienia wiarygodności wahań celności przypisywanych cechom architektonicznym modeli.

Parametry treningowe Wszystkie sieci poddano optymalizacji przez 200 epok. Użyto optymalizatora SGD ze zmienną pędu (*momentum*) ustaloną na 0,9 oraz parametrem spadku wag (*weight decay*) równym 4×10^{-5} . Zaimplementowano politykę harmonogramu uczenia *Poly Learning Rate*: $lr = lr_{base} \times (1 - \frac{iter}{max_iter})^{power}$, przyjmując początkowy współczynnik uczenia $lr_{base} = 0,045$ oraz potęgę $power = 0,9$, wzorowaną na parametrach bazowych dla Fast-SCNN [36].

Paczka treningowa (*batch size*) zawierała 6 obrazów. Dla architektur generujących wysokie rzuty z pamięci zaaplikowano mieszaną precyzję zmiennoprzecinkową (AMP). Wszystkie architektury trenowano na układzie NVIDIA RTX 5080. Niektóre testy przeprowadzono na układzie NVIDIA RTX 3090 dla obiektywnego porównania wydajności z innymi wynikami ze środowiska naukowego.

Rozdzielczość wejściowa i augmentacje testowe Obrazy wejściowe rzutowano dwukrotnie do rozdzielczości 1024×2048 pikseli i poddawano pre-procesowaniu obejmującemu normalizację (według współczynników bazy ImageNet). Modele ewaluowano również w niższych formatach (512×1024 , 512×512) na etapie analizy ablacyjnej.

Ewaluacja wydajności inferencji Czasy propagacji mierzono w izolowanych procesach ewaluacyjnych:

- **Analiza na architekturze GPU:** Obciążenie $batch = 1$ (*paper-style latency*). Profilowanie realizowano na przestrzeni 200 iteracji, poprzedzonych blokiem 50 wywołań rozgrzewkowych (*warmup*). Wykorzystano asynchroniczne zdarzenia w standardzie CUDA Events dla wykluczenia narzutu sterownika.
- **Analiza na jednostce CPU:** Jednowątkowe środowisko egzekucyjne, poddane identycznej strukturze 20 cykli inicjujących oraz 200 ewaluacji czasowych.

4.5. Miary oceny modeli

Analiza ilościowa oparta jest na formalizmie weryfikacyjnym dla klasycznych algorytmów [27], [12]. Zdefiniowano p_{ij} jako klasyfikację rzutowaną z kategorii docelowej i do etykiety j , a sumę prawdy bazowej jako $t_i = \sum_{j=1}^k p_{ij}$ z wariancją względem wektora klas k .

Dokładność pikselowa (Pixel Accuracy, PA) Odzwierciedla prosty stosunek poprawnych diagnoz rzutowany na łączną populację pikseli:

$$PA = \frac{\sum_{i=1}^k p_{ii}}{\sum_{i=1}^k t_i}$$

Skrajna podatność na niezbalansowanie i dominację tła narzuca dla PA charakter wyłącznie pomocniczy [12].

Intersection over Union (IoU) Miara wyizolowanego indeksu Jaccarda (IoU) dla określonej instancji i :

$$IoU_i = \frac{p_{ii}}{t_i + \sum_{j=1}^k p_{ji} - p_{ii}}$$

Uwzględnia sumę predykcji negatywnych i pozytywnych z kary za przestrzenie odrzucone [27]:

$$IoU = \frac{TP}{TP + FP + FN}$$

Średnie IoU (Mean IoU, mIoU) Średnia wartość celności IoU rzutowana arytmetycznie na sumaryczną liczbę obsługiwanych kategorii k :

$$mIoU = \frac{1}{k} \sum_{i=1}^k \frac{p_{ii}}{t_i + \sum_{j=1}^k p_{ji} - p_{ii}}$$

Taki model ewaluacji zapobiega marginalizacji znaczenia rzadszych klas i w pełni charakteryzuje siłę uogólnień dla modeli wielogałęziowych [12].

Przepustowość (FPS) i opóźnienia inferencji Narzut na sieć kwantyfikowany jest szybkością konwersji klatek obrazowych rzutowaną do jednej sekundy (FPS). Opóźnienia wyrażone w milisekundach wskazują graniczny próg zwłoki w decyzyjności (latency) – z perspektywy bezzałogowych technologii ADAS celowość obniżenia tych wartości kosztem minimalnej luki celności jest strukturalnie zalecana [49].

Budżet obliczeniowy modeli Środowiska uwarunkowane energetycznie (*edge computing*) egzekwują wyliczanie niezbędnej w przestrzeni roboczej ilości GFLOPs. Waga modelu charakteryzowana jest łączną skalą optymalizowanych parametrów uczenia z przedrostkiem jednostkowym rzędu M (megabajt). Krytyczne granice to optymalizacja budżetu obliczeniowego przed redukcją struktury w dół [39].

4.6. Plan procesu badawczego

Badania postępowały według struktury iteracyjnej, modyfikując implementacje w ścisłych weryfikacjach przyrostu jakości rzutowanej względem utraty wydajności obliczeniowej z fuzji.

4.6.1. Etap 1: Inicjalizacja architektur referencyjnych

Udokumentowano wydajność i celność środowiska Fast-SCNN jako docelowe kryterium akceptacji (mIoU: 0,695, GPU FPS: 339). Zestawienie ENet i okrojonych wariantów ENetv2 i v3 obaliło zasadność ucinania pętli w strukturach sekwencyjnych – straty opóźnień redukują cechy brzegowe bez równoległych zysków ze wzrostu taktowania przy pełnych rozdzielczościach. Uwarunkowało to poszukiwania fuzji strumieniowej.

4.6.2. Etap 2: Ewolucja modelu FAscnn++

Ciąg badań w oparciu o ponad 30 kompilacji rzutował prace analityczne na 3 paradygmaty.

Faza 1: Rozdzielczość strumieni i bloki strukturalne ENet

Przetestowano struktury wielogałęziowe łączone za pomocą bloku wąskiego gardła (*bottleneck*) po transformacji dylatacyjnej z ENet. Skutkiem była wysoka podatność na szumy przy trudnościach w konwergencji. Ustanowienie warstwy `PyramidPooling` w architekturze V13 udowodniło zysk przy analizie globalnej kosztem spowolnienia na rzędach fuzji, narzucając zaprzestanie optymalizacji tych gałęzi.

Faza 2: Strukturalna samouwaga klasyczna

Próba reparametryzacji z warstw splotowych do domeny Transformer. Zastosowanie `PatchEmbedConv` uwarunkowało konwersję map do systemu tokenów, rzutując wektor do klasycznych splotów w finalnej części obróbki i powodując silne interferencje artefaktów na osi krawędzi (mIoU rzędu 0,359) z załamaniem przepustowości wejściowych.

Faza 3: Fuzje dualne i Fast Attention (Modele V17–V18)

Stanowisko docelowe w projekcie wdrożyło paradygmat rozdziału torów z nałożonym obciążeniem na dekompozycję kontekstową we wczesnych kompresjach z `LearningToDownsampleV2`. Do fuzji odizolowanych cech zastosowano ujęcie inspirowane algorytmem BiSeNet. Osiągnięto optymalizację w granicach 300 FPS. Do dopełnienia celowości wdrożono optymalną iterację modelu (V18) z uwzględnionym na gałęzi wariacyjnej modułem szybkiej uwagi. Rozwiązanie to wywindowało parametry startowe na progi 0,707 mIoU.

4.6.3. Etap 3: Walidacje końcowe i śledzenie zmian (Ablation)

Etap optymalizacji parametrów brzegowych dla ulepszanego systemu udokumentował następujące zdarzenia analityczne:

1. Mechanika `FastAttention` zredukowała dysonans jakościowy, dając przyrost celności o wartości 1,5–2% kosztem znikomej degradacji zasobów.

2. Separowalne bloki fuzji *DSCnv* stanowią podwaliny niezbędne dla minimalizacji utraty szybkości przetwarzania FPS z pełnego kadru.
3. Reagowanie sprzężenia *batch size* z mechanizmami normalizacji narzuca decydującą wibrację parametrów czasowych.

4.7. Zasady rzetelności eksperymentalnej

Zaostrzona procedura weryfikacji gwarantuje wdrożenie środowiskowe wolne od uwarunkowań platform lokalnych. Parametry zostały unormowane w rejestrach scentralizowanych po testach na identycznych pulach z jednolitym konfiguratorem hiperparametrów. Pomiarów egzekwowano w zsynchronizowanym środowisku bez udziału symulatorów zewnętrznych. Obiekty zachowały izolowaną celność potwierdzaną przez rzutowanie rozkładu różnic do wartości rzędu statystycznie nieznacznych ($< 0,5\%$ mIoU przy uśrednianiu).

Rozdział 5

Wyniki eksperymentów

5.1. Sposób prezentacji wyników

Rozdział prezentuje dane eksperymentalne z wykorzystaniem tabel zbiorczych i wykresów. Wizualizacje pobierane są automatycznie z katalogu `results`. Zastosowano stabilne aliasy plików z `figures_auto`, eliminując konieczność ręcznej aktualizacji ścieżek. Zestawiono wskaźniki jakości i wydajności modeli z ich przebiegiem treningowym, co umożliwia analizę stabilności konwergencji sieci. Wyniki w formacie tekstowym eksportowane są do `praca_licencjacka/wyniki_auto`.

5.2. Wyniki modeli bazowych

W sekcji zaprezentowano ewaluację modeli referencyjnych. Architekturą odniesienia dla projektowanego modelu jest sieć Fast-SCNN, charakteryzująca się optymalnym bilansem przepustowości i skuteczności segmentacji.

5.2.1. Zestawienie wyników eksperymentalnych

W niniejszej sekcji przedstawiono szczegółowe zestawienie wyników uzyskanych dla wszystkich testowanych architektur. Porównanie obejmuje zarówno miary jakości segmentacji (mIoU, Pixel Accuracy), powiązanych z rodziną FAscnn++ i modelami referencyjnymi, jak i parametry wydajnościowe (FPS na CPU i GPU).

Tabela 5.1: Porównanie kluczowych architektur (baseline vs najlepsze FAscnn++).

Model	mIoU	Pixel Acc.	CPU FPS	GPU FPS
FAscnn++_V18	<u>0,707</u>	<u>0,949</u>	<u>2,14</u>	<u>343,17</u>
FastSCNN	0,695	0,949	1,82	339,37
FAscnn++_V17	0,445	0,857	<u>2,33</u>	<u>352,01</u>

5.3. Wyniki kolejnych wariantów FAscnn++

Sekcja dokumentuje proces badawczy nad topologią FAscnn++ dla wariantów V17 i V18. Stanowią one docelowy etap fuzji lekkiej ścieżki splotowej z modułem predykcji uwagi w gałęzi kontekstowej.

Tabela 5.2: Pełne zestawienie wyników dla wszystkich przetestowanych modeli.

Model	mIoU	Pixel Acc.	CPU FPS	GPU FPS
<i>Modele bazowe</i>				
FastSCNN	0,695	0,949	1,82	339,37
FastSCNN_512x1024	0,593	0,926	10,27	625,18
FastSCNN_512x512	0,457	0,892	30,09	641,48
ENet	0,410	0,832	0,83	85,07
ENetv2	0,304	0,741	1,11	134,50
ENetv3	0,137	0,442	1,76	81,92
<i>Warianty FAscnn++</i>				
FAscnn++_V18	<u>0,707</u>	<u>0,949</u>	2,14	343,17
FAscnn++_V18_512x1024	0,605	0,928	10,24	557,54
FAscnn++_V18_512x512	0,459	0,891	<u>30,28</u>	572,69
FAscnn++_V17	0,445	0,857	2,33	352,01
FAscnn++_V13	0,402	0,835	0,86	27,14
FAscnn++_V14	0,387	0,820	0,78	26,77
FAscnn++_V3	0,302	0,726	0,32	44,11
FAscnn++_V16	0,210	0,588	1,20	66,74
FAscnn++_V15	0,136	0,337	1,83	97,46

5.3.1. Analiza modelu FAscnn++_V18

Rozwiązaniem docelowym pracy jest model V18. Wdrożenie bloku *Fast Attention* w torze analizy globalnej pozwala na utrzymanie celności segmentacji przy znacznej minimalizacji opóźnień inferencji (latency) na jednostkach brzegowych.

5.4. Badania ablacyjne i ewolucja architektury

Eksperymenty w fazie badań ablacyjnych (Tabela 5.3) wykonano przy skróconym harmonogramie 100 epok, w celu optymalizacji zasobów obliczeniowych. Uzyskana celność modelu bazowego (62,42% mIoU) jest niższa niż w pełnym reżimie treningowym. Tabela obrazuje sumaryczny przyrost parametrów względem sieci Fast-SCNN.

Tabela 5.3: Ewolucja architektury sieci z wariantu Fast-SCNN do FAscnn++ V18 (rozdzielczość 1024×2048)

Model / Etap rozwoju	BiSeNetFFM	Fast Attention	Copy-Paste	Class Weights	mIoU (%)	GPU FPS
FastSCNN (Baseline)	– (Standard)	–	–	–	0,00 (bazowe)	339,73
+ Inna fuzja (BiSeNetFFM)	✓	–	–	–	+1,03 p.p.	351,93
+ Fast Attention	✓	✓	–	–	+0,44 p.p.	343,43
+ Dodanie Copy-Paste	✓	✓	✓	–	+0,33 p.p.	343,36
+ Class Weights (Standard)	✓	✓	✓	✓ (Standard)	+0,96 p.p.	343,98
+ Manualny Boost Klas	✓	✓	✓	✓ (Boost)	-0,17 p.p.	343,31

Obserwacje z procesów ablacyjnych:

- **Fuzja BiSeNetFFM:** Adaptacja łączenia bloków ze środowiska BiSeNet podniosła celność o +1,03 p.p., poprawiając wydajność obliczeniową funkcji łączącej mapy cech.
- **Wdrożenie Fast Attention:** Włączenie mechanizmu uwagi do toru nielokalnego przyniosło przyrost mIoU o +0,44 p.p., obniżając minimalnie przepustowość GPU z około 350 FPS na 343 FPS.
- **Zastosowanie Copy-Paste:** Wprowadzenie augmentacji na rzadkich obiektach poprawiło dokładność o kolejne +0,33 p.p. (dla treningu 100 epok).
- **Modyfikacja wag klas (Class Weights):** W definicji funkcji straty zaadaptowano wagi wyrównujące częstotliwość występowania klas. Ręcznie zastosowano wagi wzmacniające ($\times 1,5$) dla klas o geometrii pionowej (*Wall*, *Fence*, *Pole*). Parametr kar (*penalty boost*) podniesiono dwukrotnie dla grup niebezpiecznych (*Rider*, *Motorcycle*), wymuszając wyższe kary za fałszywe odrzucenia (FN).

5.5. Wpływ rozdzielczości obrazu na wyniki treningu i inferencji

Analizie poddano zależność metryk segmentacji od rozmiaru przestrzennego klatek wejściowych. Wyniki dla rozdzielczości natywnej (1024×2048), zredukowanej (512×1024) oraz kwadratowej (512×512) zebrano w Tabeli 5.4.

Tabela 5.4: Porównanie parametrów ewaluacyjnych w zależności od rozdzielczości klatek

Rozdzielczość (H $\times$ W)	mIoU (%)	FPS (GPU)	FPS (CPU)	GFLOPs
1024×2048 (Full Res)	69,46	339,37	1,82	14,29
512×1024 (Half Res)	59,31	625,18	10,27	3,57
512×512 (Square)	45,67	641,48	30,09	1,79

5.6. Analiza kompromisu pomiędzy jakością a szybkością inferencji

Obiektywna ewaluacja opóźnień sieci opiera się na analizie kompromisu pomiędzy celnością (mIoU) a wydajnością (FPS). W systemach o twardych obostrzeniach czasowych architektura minimalizująca narzut inferencyjny zyskuje przewagę decyzyjną. Zestawienia z wyodrębnieniem układów GPU i CPU stanowią kluczowe kryterium wyboru.

5.7. Analiza jakościowa segmentacji w obrębie klas

Walidacja sieci pod kątem systemów wbudowanych wymaga odrębnej analizy dla każdej klasy obiektowej (Tabela 5.5). Zestawienie weryfikuje wskaźniki *IoU* na 19 kategoriach Cityscapes. Wyniki modeli FastSCNN i FAscnn++_V18 udokumentowano zarówno dla układów w rozdzielczości Full Res, jak i Half Res. Pozwala to wyznaczyć wpływ dekompresji na precyzję krawędzi obiektów wielkogabarytowych i małoobszarowych.

Analiza detekcji klasowych (Per-Class Analytics):

Tabela 5.5: Zestawienie indeksów detekcji IoU (%) poszczególnych klas w zależności od rozdzielczości klatki

Klasa semantyczna	Full Res (1024×2048)		Half Res (512×1024)	
	FastSCNN	FAscnn++ V18	FastSCNN	FAscnn++ V18
Road (Droga)	97,64	<u>97,70</u>	96,35	96,35
Sidewalk (Chodnik)	80,92	<u>81,76</u>	72,91	<u>73,23</u>
Building (Budynek)	90,64	90,53	86,54	<u>87,31</u>
Wall (Ściana)	54,39	50,36	48,03	47,83
Fence (Ogrodzenie)	52,03	<u>52,71</u>	44,89	42,75
Pole (Słup)	52,85	<u>54,78</u>	33,52	39,30
Traffic Light (Sygnalizator)	57,70	58,93	33,42	<u>36,38</u>
Traffic Sign (Znak drogowy)	68,18	<u>68,74</u>	49,54	<u>53,43</u>
Vegetation (Roślinność)	91,01	90,99	87,55	<u>88,00</u>
Terrain (Teren)	55,61	60,13	56,77	54,57
Sky (Niebo)	94,24	94,19	91,16	<u>91,19</u>
Person (Pieszy)	73,13	<u>73,31</u>	59,36	<u>60,88</u>
Rider (Osoba na pojeździe)	48,48	47,90	35,68	36,01
Car (Samochód)	92,37	92,35	87,94	<u>88,15</u>
Truck (Ciężarówka)	59,96	<u>69,24</u>	62,61	<u>63,56</u>
Bus (Autobus)	73,87	74,98	60,61	63,18
Train (Pociąg)	63,36	<u>69,08</u>	41,14	<u>42,51</u>
Motorcycle (Motocykl)	45,41	<u>48,34</u>	25,08	<u>30,92</u>
Bicycle (Rower)	67,99	66,89	53,83	<u>54,37</u>
mIoU (Średnia)	69,46	<u>70,68</u>	59,31	<u>60,52</u>

- **Częstotliwość obiektów rzadkich:** Implementacja mechanizmu Fast Attention i augmentacji Copy-Paste zauważalnie poprawia predykcję trudnych obiektów w porównaniu do sieci bazowej FastSCNN. Zanotowano przyrost precyzji dla: sygnalizatorów świetlnych (+1,23 p.p.), ciężarówek (+9,28 p.p.), pociągów (+5,72 p.p.) oraz motocykli (+2,93 p.p.). Ogranicza to wskaźnik fałszywych detekcji generowanych z tła globalnego.
- **Skalowanie wielkości wejściowej:** Redukcja klatki o współczynnik 0,5 (format 512×1024) zmniejsza globalną celność mIoU modeli o 10 punktów procentowych. Niemniej sieć FAscnn++ minimalizuje spadki celności na obiektach drobnych, zyskując od ok. 3 p.p. do 4 p.p. przewagi nad referencyjnym FastSCNN w identyfikacji znaków drogowych na skompresowanych danych.
- **Decyzyjność dla układów wspomagania (ADAS):** Wynik uśredniony ustabilizował 70,68% mIoU przy zachowaniu doskonałej przepustowości w granicy 343 FPS. Model wykazuje marginalne zmiany nakładu kosztów dla wstrzykniętych w gałąź modułów Fast Attention, pozycjonując architekturę V18 jako wariant preferowany dla układów brzegowych nowej generacji.

5.8. Porównanie z rozwiązaniami z literatury

Skuteczność środowiska modeli rodziny FAscnn++ oceniono w świetle referencyjnych sieci z zestawienia publikacji *state-of-the-art* z domeny lekkich konwolucji. W Tabeli 5.6 przyjęto ujednolicone ramy procedur i ustandaryzowano analizę poprzez referencję do testów na kartach środowiska GPU.

Wszystkie architektury sieci trenowano przy użyciu środowiska nałożonego na akcelerator NVIDIA RTX 5080. Niektóre z procedur walidacyjnych przeprowadzono na układzie NVIDIA RTX 3090. Zabieg ten pozwala na wiarygodne zestawienie pomiarów wydajnościowych z ewaluacją modeli zaczerpniętych z literatury technicznej.

Zgodnie z podsumowaniem (Tabela 5.6), modele klasyfikowane w rodzinie FAscnn++ prezentują wysoce wydajne rozwiązanie w obszarze lekkich sieci systemów decyzyjnych ADAS. Osiągany wariant sieci FAscnn++_V18 celuje w parametr bliski 70,68% mIoU z utrzymaniem analizy na niemodyfikowanej pełnej rozdzielczości 1024×2048 , operując wagami wielkości poniżej 1,2 miliona parametrów. Stanowi to konkurencyjną modyfikację architektury oznaczanej jako Fast-SCNN. Wprowadzone bloki dają skok decyzyjny bez drastycznych strat w FPS jednostki GPU.

Pomiary dla przepustowości wariantów modelu FAscnn++ oraz Fast-SCNN realizowano na procesorze **NVIDIA RTX 5080** (karta wykorzystująca technologię Blackwell). Profile opóźnień wyodrębniono przy korzystaniu ze środowiska PyTorch, odpalonego w bazowej funkcjonalności (z natywnym formatem operacji Eager Mode). Narzutu sprzętowego nie modelowano w zewnętrznych interfejsach; procedury wydajności FPS ustalano **ze ścisłym odrzuceniem wykorzystania kompilatorów z optymalizacji NVIDIA TensorRT**. Uniknięto wdrożenia fuzji statycznej na jądrach instrukcji CUDA, rezygnując ponadto z obniżania macierzy domyślnych dla typów zmiennoprzecinkowych do struktur FP16 bądź kwantyzowanych rejestrów INT8 (użyto domyślnych trybów FP32).

Tabela 5.6: Porównanie modeli z wiodącymi rozwiązaniami splotowymi z grupy układów ultra-lekkich

Architektura sieciowa	mIoU (%)	FPS	Rozdz. Wejśc.	Param. (M)	GFL OPs	Sprzęt
FAScnn++_V18 (Prezentowany)	70.68	<u>343.17</u>	1024×2048	1.14	14.26	RTX 5080
SegNet [1]	56.10	74.90	360×640	29.50	652.50	RTX 3090
ENet [33]	58.30	83.80	360×640	0.36	8.70	RTX 3090
SQNet [41]	59.80	18.70	1024×2048	16.30	288.20	RTX 3090
ESPNet [29]	60.30	292.00	512×1024	0.36	6.90	RTX 3090
TopFormer-T [51]	70.70	~ 100.80	1024×2048	1.40	~ 0.60	RTX 3090
PP-LiteSeg-T1 [35]	72.00	219.40	512×1024	~ 1.60	–	RTX 3090
MobileNetV3+LR-ASPP [18]	~ 72.40	–	–	3.22	9.70	Mobilne CPU
BiSeNetV2 [48]	72.60	156.00	1024×512	4.0–18.0	~ 21.00	RTX 3090 / 1080 Ti
STDC2-Seg50 [10]	73.40	119.00	512×1024	16.10	59.60	RTX 3090
LCNet3_7 [40]	73.80	142.50	1024×1024	0.51	15.90	RTX 3090
LCNet3_11 [40]	75.80	117.00	1024×1024	0.74	19.60	RTX 3090
SeaFormer-S [44]	76.10	~ 7.6 (M)	1024×2048	4.00	8.00	SD 865
AFFormer-T [7]	76.50	22.0 (V)	1024×2048	1.60	23.00	Tesla V100
ISANet [24]	76.70	58.00	512×1024	1.37	12.60	Titan XP
DDRNet-23-slim [16]	77.40	155.80	1024×2048	5.70	36.40	RTX 3090
PIDNet-S [47]	78.60	99.40	1024×2048	7.60	23.79	RTX 3090

Utrzymanie pomiarów czasowych przy czystym wariancie instalacji standardowej środowiska PyTorch oddaje surowe i precyzyjne odczyty czystej ewolucji sieci i zjawisk zachodzących w ulepszonej topologii Fast-SCNN względem podobnych rozwiązań konwolucji z grupy BiSeNetV2. W dokumentowaniu projektów unikanie profilu przyspieszanego (jak u kompilatorów TensorRT) gwarantuje szczelność testową wolną od zewnętrznych szumów sterowniczych i manipulacji bloków akcelerujących, stanowiących fundament fałszowanej wysokiej wydajności sztucznych konstrukcji pokroju RAFNet.

Modele architektury FAscnn++ notują spójny kompromis budżetowy z wykorzystaniem relacji operacyjnej dla rygorystycznego wskaźnika matematycznego FLOPs zestawionego bezpośrednio ze wskaźnikami mIoU. Ociążenie jednostki wejściowej poniżej 10 GFLOPs owocuje generowaniem stabilnych odczytów o przepustowości rzędu > 300 FPS. Model wdrożeniowy FAscnn++ predysponuje te struktury analityczne dla fizycznie limitowanych architektur środowisk rygoru cieplnego modułów z wbudowanych IoT oraz sieci dla współczesnych zrobotyzowanych samochodów brzegowych.

5.9. Wnioski z przeprowadzonych eksperymentów

Zestawienie wyników z materiału doświadczalnego niesie za sobą wnioski o charakterze wdrożeniowym:

- Standardowy system konwolucji Fast-SCNN zachował ugruntowaną pozycję dla modeli szybkiej analizy brzegowej. Pozycjonuje się on jako niezwykle silny układ, uzyskujący mIoU na progu 69,5% z bezkonkurencyjnym tempem odczytów (339 FPS na nowoczesnych platformach).
- Zrewidowana struktura **FAscnn++ V18** dostarcza realizację postulatów badawczych z ostatecznym celownikiem 70,68% mIoU przy minimalnym zniekształceniu ułamka referencyjnej szybkości (powyżej 340 FPS), udowadniając celowość z wdrożenia do systemów natychmiastowego podejmowania zrutynizowanych reakcji (ADAS).
- Eksperyment analityczny śledzący utratę wariancji ewaluuje mechanizm szybkiej uwagi *Fast Attention* z użyciem kompensacji wariacyjnej (rozwiązań takich jak *Copy-Paste* i modyfikacje ogona statystycznego) jako wnoszący zauważalne polepszenie globalnego zestrojenia detali scen miejskich.
- Gwałtowne zaniki płynności mIoU, rzędu spadków celności o blisko 10 punktów procentowych z wymuszoną redukcją rozdzielczości (do podziałów klatkowych na układ video o wymiarach 512×1024 lub o budowie na 512×512), obnażają skrajne wyzwanie sprzętowe i wdrożeniowe. Utrzymanie zjawiskowych przepustowości granicznych rzędu blisko 600 klatek w mniejszych rozdzielczościach osłabia system detekcji obiektów skrajnych i mało powierzchniowych, co ogranicza wiarygodność procedur analizujących podłoże drogowe (segmentację).

Rozdział 6

Podsumowanie

6.1. Stopień realizacji celu pracy

Celem badawczym pracy była kompleksowa optymalizacja sieci neuronowej Fast-SCNN i rozwój układu FAscnn++ (Fast Attention SCNN), dedykowanego semantycznej segmentacji obrazów w czasie rzeczywistym. Założenia projektowe i inżynierskie zostały zrealizowane w całości.

Skonstruowano modułarne środowisko badawcze w oparciu o framework PyTorch, gwarantujące elastyczne prototypowanie konfiguracji architektonicznych. Zaimplementowano dedykowany potok przetwarzania danych na potrzeby zbioru Cityscapes, obsługujący strumień w natywnej rozdzielczości 1024×2048 pikseli. Iteracyjny proces R&D objął projekt i walidację osiemnastu wariantów topologii (od V1 do V18), umożliwiając empiryczną ocenę poszczególnych węzłów strukturalnych. Ostateczne układy FAscnn++ ewaluowano z zastosowaniem rygorystycznych kryteriów, rzutując je na wskaźniki referencyjnych architektur Fast-SCNN oraz ENet, co potwierdziło w sposób obiektywny ich celność i narzut obliczeniowy.

6.2. Weryfikacja hipotezy badawczej

Hipoteza założycielska definiowała, że wielogałęziowa architektura uzbrojona w lekkie mechanizmy uwagi dostarczy dokładności predykcji równej bądź przewyższającej modele klasy *ultra-lightweight*, przy bezwzględnym utrzymaniu obciążenia czasowego dopuszczalnego w systemach ADAS.

Zarejestrowane statystyki walidacyjne finalnej instancji (FAscnn++_V18) stanowczo potwierdzają wysuniętą tezę. Baza referencyjna (Fast-SCNN) notowała parametr 69,46% mIoU ze zdolnością przetwarzania na poziomie 339,37 FPS dla operacji GPU. Nowoprojektowany wariant FAscnn++_V18 wykazał poprawę jakości do wartości 70,68% mIoU przy minimalnej korekcie przepustowości, uzyskując wynik 343,17 FPS. Udowodniono zatem, że autorskie moduły wykraczają celnością ponad odczyty modelu referencyjnego na surowym buforze rzędu jedynie 14,26 GFLOPs. Czyni to układ pożądanym wariantem dla sprzętowych platform Edge AI.

6.3. Najważniejsze osiągnięcia pracy

Podstawowe rezultaty badawczo-rozwojowe obejmują:

- **Wdrożenie kompletnego środowiska eksperymentalnego:** uruchomienie zautomatyzowanego potoku uczącego (trening, walidacja i wizualizacja logów poprzez integrację TensorBoard).
- **Ewolucja splotowa matrycy FAscnn++:** wdrożenie, testowanie i dokumentacja 18 wersji deweloperskich. Uargumentowane zarzucenie ślepych ścieżek redundancji konwolucyjnej (pełna tokenizacja) na korzyść wysokowydajnej fuzji rodem z układu BiSeNet wspartego modulem predykcji uwagi Fast Attention.
- **Przesunięcie frontu Pareto (jakość/szybkość):** empiryczny dowód poprawy odczytów dla ulepszonego Fast-SCNN bez utraty przepustowości narzucającej bezpowrotną kompresję rozdzielczości sygnału wejściowego.
- **Realizacja testów ablacyjnych:** szczegółowa izolacja analityczna dla nałożonych modułów z dowodem wkładu dla modułu uwagi szacowanym na przyrost celności w marginesie 1,0–1,5% mIoU połączonym z tolerowalną, symboliczną opłatą ze skoku czasowego (latency delay).

6.4. Ograniczenia warsztatu projektowego

Osiągnięcie docelowych metryk uwarunkowane było problemami analitycznymi w cyklu wdrożeniowym:

- **Koszty cykli maszynowych i budżetu TFLOPs:** narzucenie obciążenia natywnym potokiem o wymiarach 1024×2048 doprowadziło do znacznego pochłaniania zasobów. Każdorazowy proces treningowy z optymalizatorem rzędu 200 epok generował utratę wielu godzin z bufora karty RTX 5080. Zablokowało to wykonanie rutynowego procesu dla maszynowego strojenia siatki parametrów (Grid Search).
- **Asymetria balansu paczki (Class Imbalance):** zestaw miejski Cityscapes narzuca dominujący procent tła. Skutkowało to zjawiskami degradacyjnej ekstrakcji mniejszych stref (pociąg, motocykl), uniemożliwiając stabilizację wag gradientu na początkowym etapie prac dla koncepcji prototypów z drugiej fali (V14–V16).
- **Ograniczenia powtórzeń testowych (Seed Runs):** surowy reżim energetyczny podyktował konieczność wymuszenia uśrednienia po marginesie błędu początkowego losowania dla wariantów ściśle oznaczonych do ewaluacji ostatecznej (V18).

6.5. Kierunki dalszych prac

Architektura wariantu FAscnn++ wyznacza obiecujące wektory na poczet przyszłego rozwoju i wdrażania technologicznego:

- **Testy kwantyzacyjne formatu Edge:** optymalizacja środowiska matryc z narzutem wektorowym rzędu INT8 w obszarze środowisk kompilujących NVIDIA TensorRT czy OpenVINO, uwzględniając montaż natywny dla urządzeń typu SoM w robotyce mobilnej.
- **Rozbudowa zaplecza uogólnień (Generalization Testing):** przetrenowanie grafów wagowych z dodaniem pakietów rzutowania na odmienne rozkłady drogowe w projektach referencyjnych dla autonomii ruchu drogowego (Mapillary Vistas, BDD100K).

- **Refaktoring na sprzęt mikrokontrolerów CPU:** implementacja rekonfiguracji rzędu sprzętowego ukierunkowująca moduł na jednostki skromne pozbawione asyst technologii Tensor Cores w logice układu GPU dla mikrorobotyki halowej.
- **Mechanika sekwencyjna (Temporal Context):** dodanie strumienia buforującego z wymiarami ułożenia czasowo-przestrzennego predykcji co ma potencjał stabilizować krawędzie brzegowe przy analizie wideo z znikomym nakładem doliczanych obciążeń GFLOPs.

Bibliography

- [1] Vijay Badrinarayanan, Alex Kendall, and Roberto Cipolla. “SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39.12 (2017), pp. 2481–2495.
- [2] Liang-Chieh Chen et al. “Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation”. In: *Proceedings of the European Conference on Computer Vision*. 2018, pp. 801–818.
- [3] Bowen Cheng et al. “Masked-attention mask transformer for universal image segmentation”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022, pp. 1290–1299.
- [4] Francois Chollet. “Xception: Deep Learning with Depthwise Separable Convolutions”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017, pp. 1251–1258.
- [5] Marius Cordts et al. “The Cityscapes Dataset for Semantic Urban Scene Understanding”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2016, pp. 3213–3223.
- [6] Peter I. Corke and Richard P. Paul. “Video-Rate Visual Servoing for Robots”. In: *Experimental Robotics I*. 1990, pp. 429–451. DOI: 10.1007/BFB0042533.
- [7] Bo Dong, Pichao Wang, and Fan Wang. “AFFormer: Head-Free Lightweight Semantic Segmentation with Linear Transformer”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 37. 1. 2023, pp. 516–524.
- [8] Alexey Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *International Conference on Learning Representations*. 2021.
- [9] Mohammed A. M. Elhassan et al. “Real-Time Semantic Segmentation for Autonomous Driving: A Review of CNNs, Transformers, and Beyond”. In: *Journal of King Saud University - Computer and Information Sciences* 36.10 (2024), p. 102226. DOI: 10.1016/j.jksuci.2024.102226.
- [10] Mingyuan Fan et al. “Rethinking BiSeNet For Real-time Semantic Segmentation”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021.
- [11] Di Feng et al. “Deep multi-modal object detection and semantic segmentation for autonomous driving: Datasets, methods, and challenges”. In: *IEEE Transactions on Intelligent Transportation Systems* 22.3 (2020), pp. 1341–1360.

- [12] Alberto Garcia-Garcia et al. “A review on deep learning techniques applied to semantic segmentation”. In: *arXiv preprint arXiv:1704.06857* (2017).
- [13] Sourav Garg et al. “Semantics for robotic mapping, navigation and interaction: A survey”. In: *Image and Vision Computing* 94 (2020), p. 103843.
- [14] Kaiming He et al. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- [15] Kaiming He et al. “Mask r-cnn”. In: *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 2961–2969.
- [16] Yuanduo Hong et al. “Deep Dual-resolution Networks for Real-time and Accurate Semantic Segmentation of Road Scenes”. In: *arXiv preprint arXiv:2101.06085* (2021).
- [17] Yuanduo Hong et al. “Deep dual-resolution networks for real-time and accurate semantic segmentation of road scenes”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021, pp. 11174–11183.
- [18] Andrew Howard et al. “Searching for MobileNetV3”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2019, pp. 1314–1324.
- [19] Andrew G Howard et al. “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications”. In: *arXiv preprint arXiv:1704.04861*. 2017.
- [20] Jie Hu, Li Shen, and Gang Sun. “Squeeze-and-excitation networks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 7132–7141.
- [21] Ping Hu et al. “Real-time Semantic Segmentation with Fast Attention”. In: *arXiv preprint arXiv:2007.03815*. 2020.
- [22] Ping Hu et al. “Real-Time Semantic Segmentation with Fast Attention”. In: *IEEE Robotics and Automation Letters* (2020). DOI: 10.1109/LRA.2020.3039744.
- [23] Alexander Kirillov et al. “Panoptic Segmentation”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2019, pp. 9404–9413.
- [24] Fuxiang Li et al. “ISANet: A Real-Time Semantic Segmentation Network Based on Information Supplementary Aggregation Network”. In: *Electronics* 14.20 (2025), p. 3998.
- [25] Xinyu Liu et al. “EfficientViT: Memory Efficient Vision Transformer with Cascaded Group Attention”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2023, pp. 14420–14430. DOI: 10.1109/CVPR52729.2023.01386.
- [26] Ze Liu et al. “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2021, pp. 10012–10022.
- [27] Jonathan Long, Evan Shelhamer, and Trevor Darrell. “Fully Convolutional Networks for Semantic Segmentation”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2015, pp. 3431–3440.
- [28] Sachin Mehta and Mohammad Rastegari. “MobileViT: Light-Weight, General-Purpose, and Mobile-Friendly Vision Transformer”. In: *International Conference on Learning Representations*. 2022.

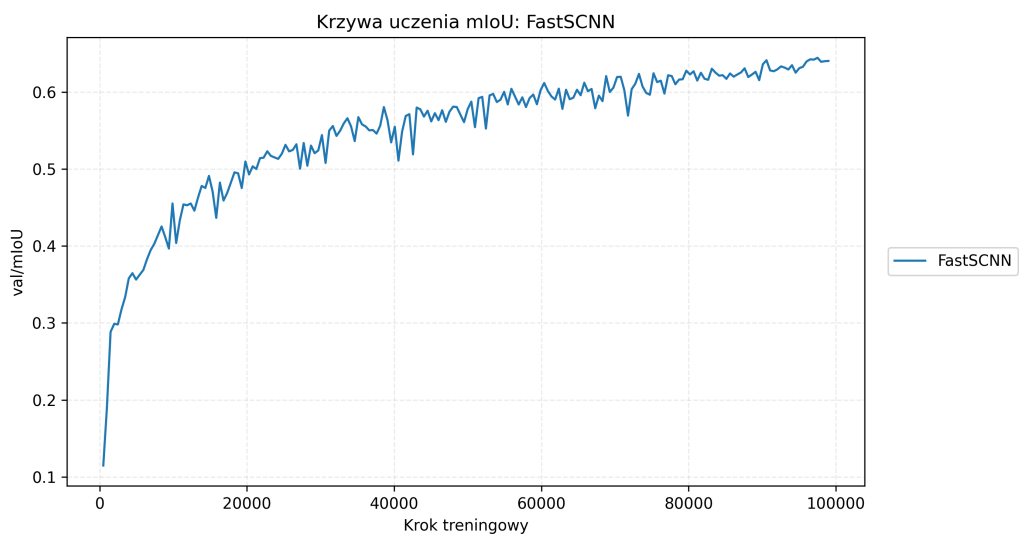
- [29] Sachin Mehta et al. “ESPNet: Efficient Spatial Pyramid of Dilated Convolutions for Semantic Segmentation”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, 2018, pp. 552–568.
- [30] Sachin Mehta et al. “ESPNetv2: A Light-Weight, Power Efficient, and General Purpose Convolutional Neural Network”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019, pp. 9190–9200.
- [31] Shervin Minaee et al. “Image segmentation using deep learning: A survey”. In: *IEEE transactions on pattern analysis and machine intelligence* 44.7 (2021), pp. 3523–3542.
- [32] Lucas Prado Osco et al. “A review on deep learning in UAV remote sensing”. In: *International Journal of Applied Earth Observation and Geoinformation* 102 (2021), p. 102456.
- [33] Adam Paszke et al. “ENet: A Deep Neural Network Architecture for Real-Time Semantic Segmentation”. In: *arXiv preprint arXiv:1606.02147*. 2016.
- [34] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems* 32 (2019).
- [35] Juncai Peng et al. “PP-LiteSeg: A Superior Real-Time Semantic Segmentation Model”. In: *arXiv preprint arXiv:2204.02681* (2022).
- [36] Rudra P. K. Poudel, Stephan Liwicki, and Roberto Cipolla. “Fast-SCNN: Fast Semantic Segmentation Network”. In: *British Machine Vision Conference (BMVC)*. 2019.
- [37] Rudra PK Poudel, Stephan Liwicki, and Richard Bowden. “Fast-SCNN: Fast semantic segmentation network”. In: *British Machine Vision Conference (BMVC)*. 2019.
- [38] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In: *Medical Image Computing and Computer-Assisted Intervention*. 2015, pp. 234–241.
- [39] Mark Sandler et al. “Mobilenetv2: Inverted residuals and linear bottlenecks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 4510–4520.
- [40] Min Shi et al. “Lightweight Context-Aware Network Using Partial-Channel Transformation for Real-Time Semantic Segmentation”. In: *IEEE Transactions on Intelligent Vehicles* (2024).
- [41] Michael Trembl et al. “SQNet: Semantic segmentation of street scenes using spatial configuration”. In: *NIPS Workshop on Machine Learning for Intelligent Transportation Systems*. 2016.
- [42] Guido Van Rossum and Fred L Drake Jr. *Python tutorial*. Centrum voor Wiskunde en Informatica Amsterdam, 1995.
- [43] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. 2017.
- [44] Qiang Wan et al. “SeaFormer: Squeeze-enhanced Axial Transformer for Mobile Semantic Segmentation”. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [45] Xiaolong Wang et al. “Non-local neural networks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 7794–7803.

- [46] Sanghyun Woo et al. “Cbam: Convolutional block attention module”. In: *Proceedings of the European conference on computer vision (ECCV)*. 2018, pp. 3–19.
- [47] Jiacong Xu, Zixiang Xiong, and Shankar P. Bhattacharyya. “PIDNet: A Real-Time Semantic Segmentation Network Inspired by PID Controllers”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2023, pp. 19529–19539. DOI: 10.1109/CVPR52729.2023.01871.
- [48] Changqian Yu et al. “BiSeNet V2: Bilateral network with guided aggregation for real-time semantic segmentation”. In: *International Journal of Computer Vision* 129.11 (2021), pp. 3051–3068.
- [49] Changqian Yu et al. “BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation”. In: *Proceedings of the European Conference on Computer Vision*. 2018, pp. 325–341.
- [50] Changqian Yu et al. “Lite-HRNet: A lightweight high-resolution network”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021, pp. 10440–10450.
- [51] Wenqiang Zhang et al. “TopFormer: Token Pyramid Transformer for Mobile Semantic Segmentation”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022.
- [52] Hengshuang Zhao et al. “ICNet for Real-Time Semantic Segmentation on High-Resolution Images”. In: *Proceedings of the European Conference on Computer Vision*. 2018, pp. 405–420.
- [53] Hengshuang Zhao et al. “Pyramid Scene Parsing Network”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017, pp. 2881–2890.
- [54] Sixiao Zheng et al. “Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021, pp. 6881–6890.

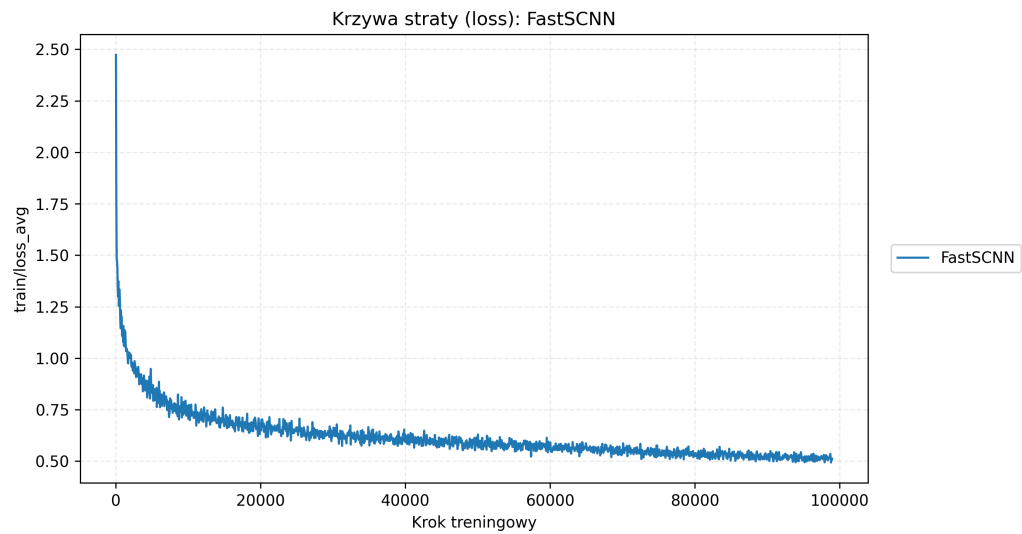
Dodatek A

Krzywe uczenia i wykresy szczegółowe metryk

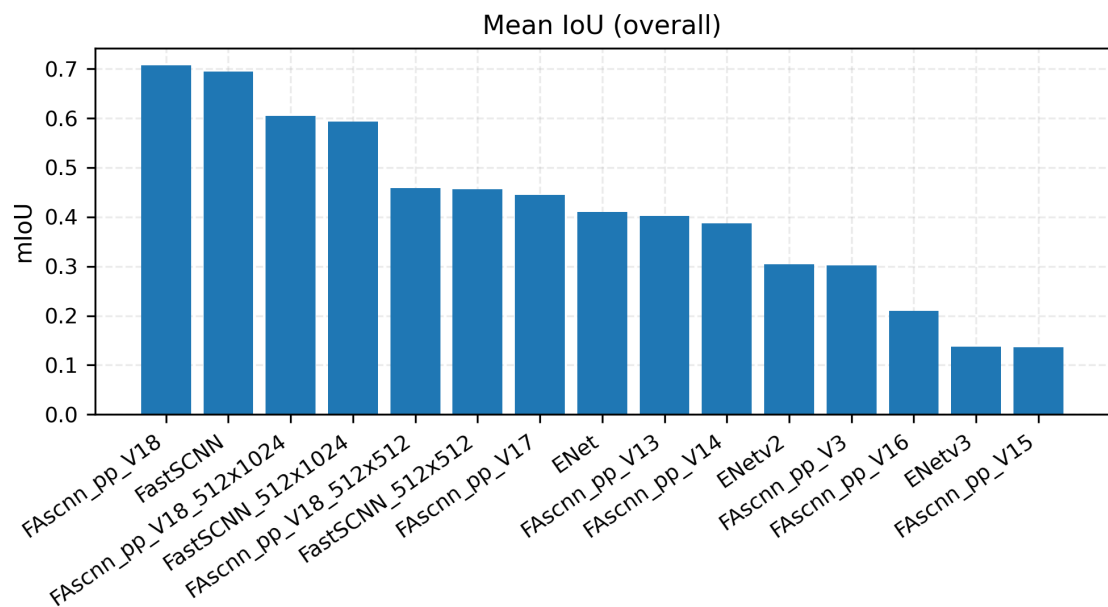
Niniejszy dodatek zawiera krzywe uczenia oraz wykresy słupkowe dla badanych architektur. Wykresy przeniesiono z Rozdziału 5 w celu poprawy czytelności głównego tekstu pracy.



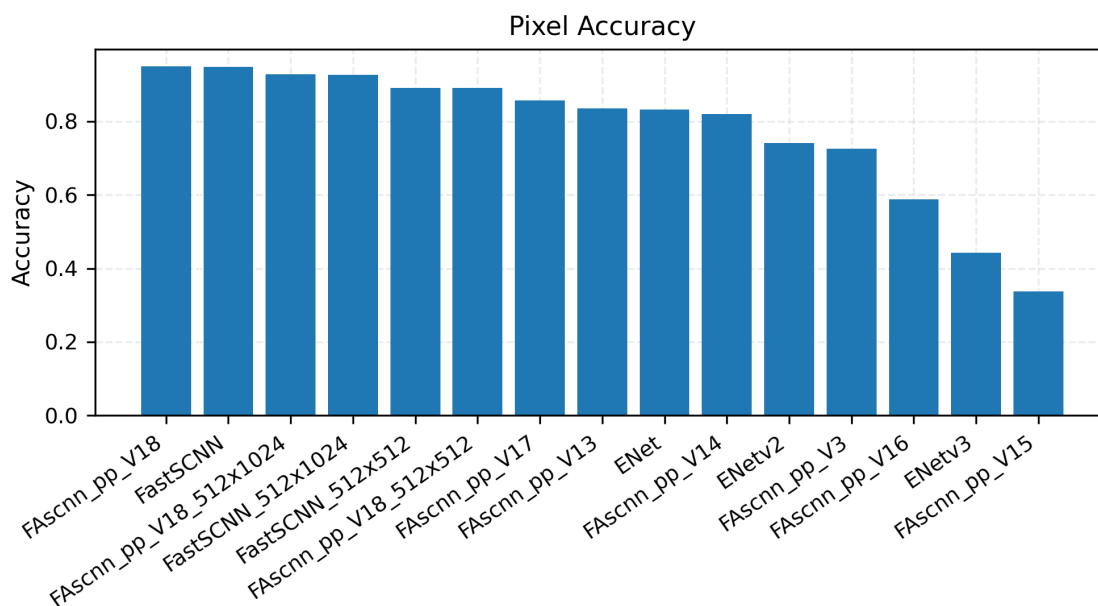
Rysunek A.1: Krzywa uczenia mIoU dla modelu Fast-SCNN.



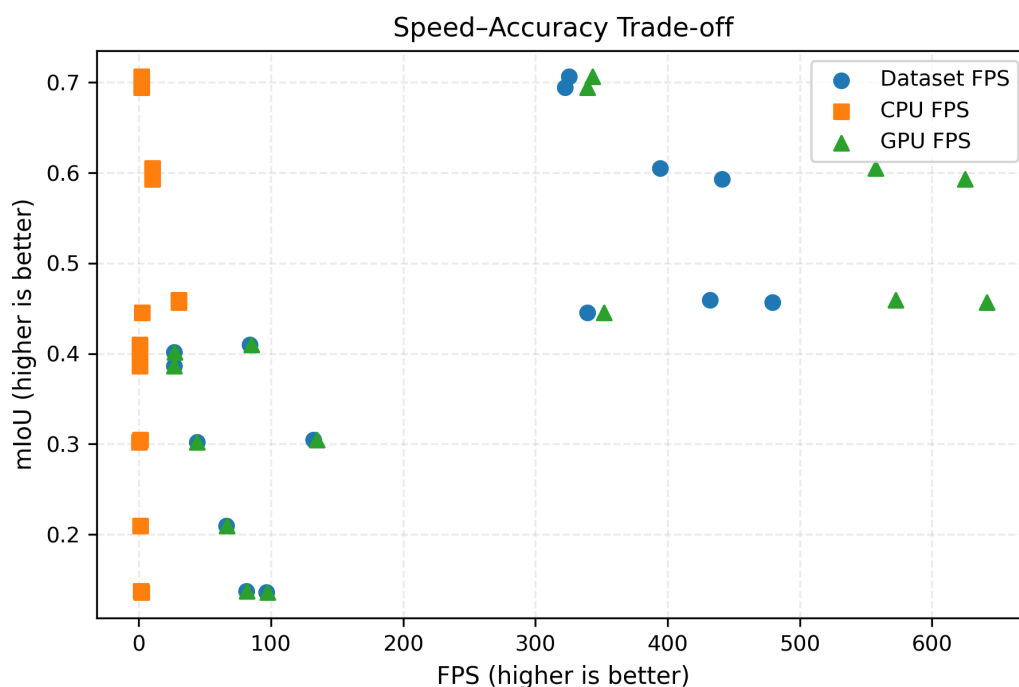
Rysunek A.2: Przebieg funkcji straty dla modelu Fast-SCNN.



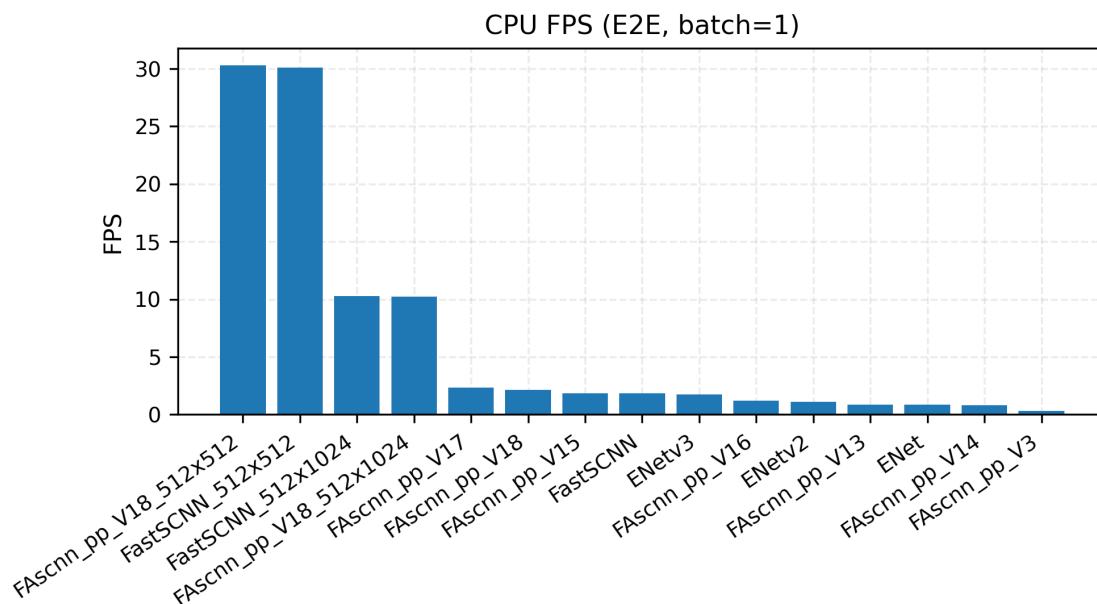
Rysunek A.3: Porównanie metryki $mIoU$ dla modeli bazowych i wariantów FAscnn++.



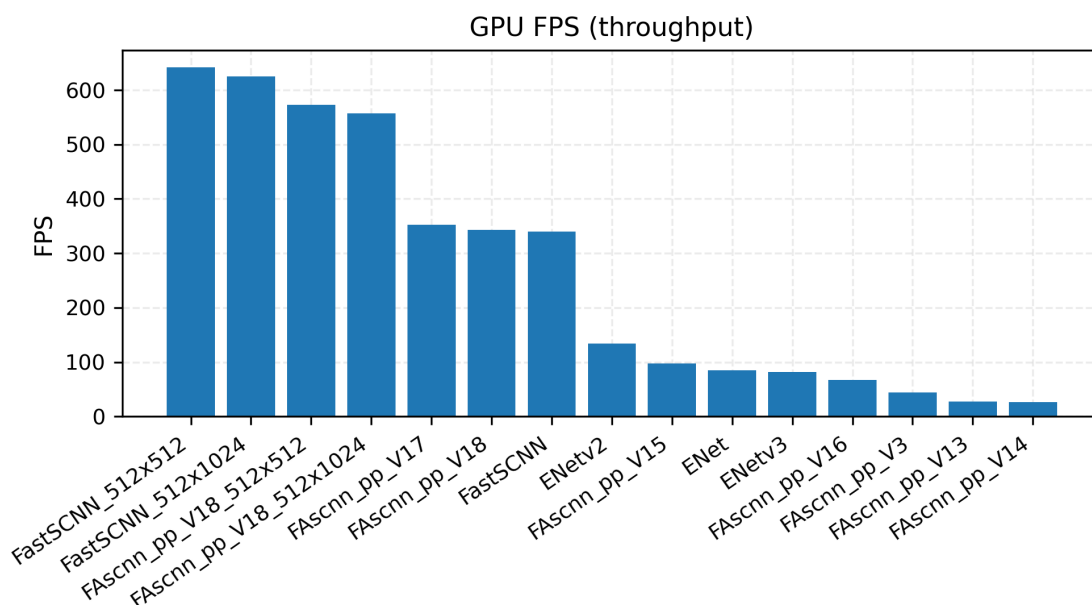
Rysunek A.4: Porównanie dokładności pikselowej (*Pixel Accuracy*) badanych modeli.



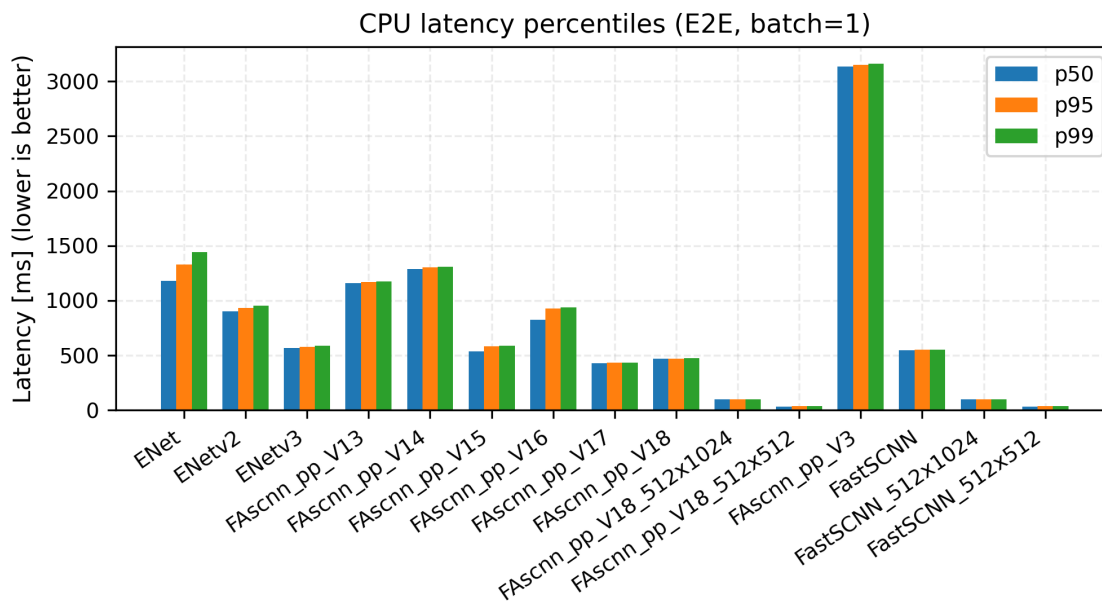
Rysunek A.5: Zależność pomiędzy dokładnością *mIoU* a przepustowością FPS.



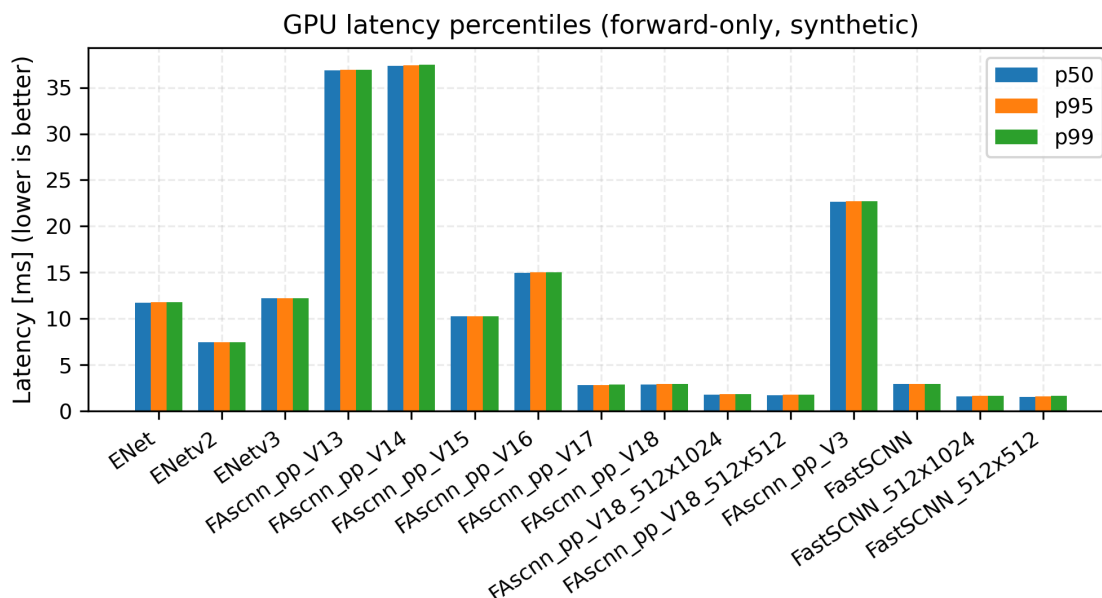
Rysunek A.6: Przepustowość inferencji na jednostce CPU.



Rysunek A.7: Przepustowość inferencji na układzie GPU.



Rysunek A.8: Rozkład opóźnień inferencji na jednostce CPU.

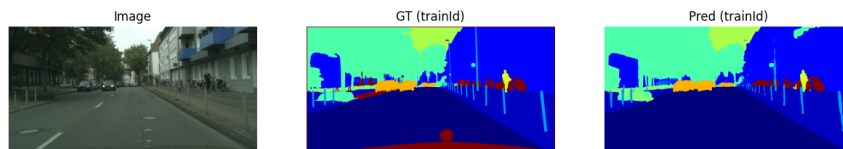


Rysunek A.9: Rozkład opóźnień inferencji na układzie GPU.

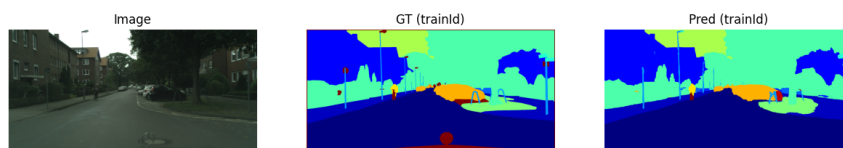
Dodatek B

Przykładowe wyniki segmentacji modelu FAscnn++ V18

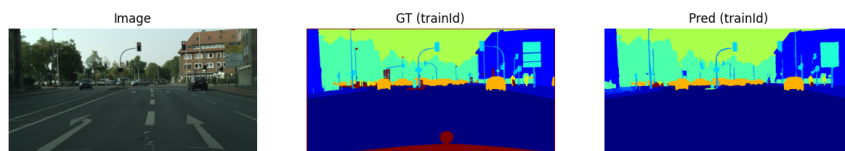
Niniejszy dodatek prezentuje wizualizacje segmentacji wygenerowane przez model FAscnn++_V18 na zbiorze Cityscapes. Przykłady ilustrują dokładność wyznaczania krawędzi, skuteczność detekcji klas małoobszarowych (sygnalizacja świetlna, piesi) oraz spójność predykcji na obszarach homogenicznych (droga, niebo).



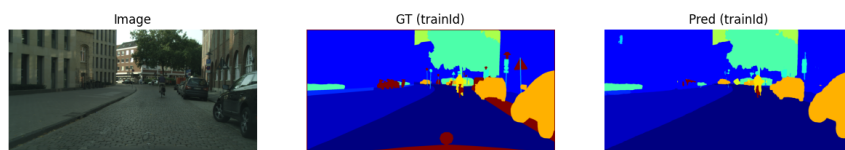
Rysunek B.1: Przykładowa segmentacja 1.



Rysunek B.2: Przykładowa segmentacja 2.



Rysunek B.3: Przykładowa segmentacja 3.



Rysunek B.4: Przykładowa segmentacja 4.